

Laboratorio 5 — Creando un servidor

Profesor: Matías Toro

Ayudantes: Benjamín Indo, Benjamín Madrid, Camilo Huerta, Daniel Ramírez, Julián Ferreira, Sebastián Bustos, Sebastián González, Tomás Méndez, Tomás Ubilla

En este laboratorio pondrán en práctica los conceptos vistos en la clase de *Programando un servidor*. Para este laboratorio utilizaremos la pagina creada en el laboratorio pasado, en caso de no haberlo hecho tendrá un template para este laboratorio en la sección de material docente en U-Cursos.

En el laboratorio anterior conectaron un servidor externo a su página. El objetivo de este laboratorio es crear su propio servidor con las solicitudes hechas en el laboratorio anterior.

P1.

Conecte su aplicación con MongoDB y cree un schema para la estructura de comentarios que usa la aplicación (o use el que definió en un lab anterior). Agregue a su esquema las siguientes restricciones:

- El largo de un comentario no puede exceder 300 caracteres.
- El largo de un comentario tiene que ser al menos 1.
- El nombre del autor de un comentario no puede estar dentro de los siguientes nombres prohibidos:
 - Huevito rey
 - Matías Toro
 - Memes es mal ramo

P2.

Cree el *endpoint* `api/threads`, que debe poder hacer:

- **GET:** obtener todos los threads creados en la página.
- **POST:** crear un thread nuevo.

Use los endpoint que ha creado para reemplazar las conexiones que tenía con anterioridad logrando mantener la página funcionando pero esta vez con un backend controlado de su parte, recuerde que deberá crear archivos json con la estructura definida en la interfaz del laboratorio anterior:

```
interface Post {  
  id: number  
  content: string  
  author: string | null  
  thread: number | null  
  parent: number | null  
  createdAt: string  
  updatedAt: string  
  likes: number  
  dislikes: number  
}
```



P3.

Cree el *endpoint* `api/thread/:id`, que debe poder hacer:

- **GET**: obtener todos los comentarios de un thread específico.
- **POST**: agregar un comentario nuevo al thread.

Siga la misma estructura de la pregunta anterior para conectar este endpoint a su frontend, reemplazando las conexiones que tenía con anterioridad.

P4.

Implemente un *middleware* para manejar los errores de su API. Este debe manejar:

- Errores de validación al crear threads.
- Errores de validación al crear comentarios.

Además tendrá que utilizar *middlewares* para manejar los errores que podrían hacer que se cayese la página durante su ejecución tales como:

- Errores de validación al crear comentarios.

P5.

Cree un *endpoint* que permita actualizar una publicación (thread o comentario) haciendo un PUT a `api/posts/:id`. Este endpoint será utilizado por el frontend para actualizar los likes y dislikes de los comentarios y threads.

Nota: El endpoint sobrescribe el objeto. Por lo tanto, si solo quiere actualizar un campo en específico, debe entregar una copia del objeto con el campo actualizado en el cuerpo de la petición al endpoint.

P6.

Para finalizar su trabajo con su proyecto, debe hacer el deploy de su aplicación web. Para esto debe:

- Crear en el servidor una carpeta para cada grupo.
- Compilar su frontend con `npm run build:ui`.
- Compilar el backend con `npm run build`.
- Subir la carpeta backend a su carpeta del servidor con `scp`.
 - Ejemplo:

```
scp -P219 -r backend fullstack@fullstack.dcc.uchile.cl:alanbrito/..
```

- Crear un archivo `.env` con las configuraciones de producción. Elija un puerto único de grupo (entre los asignados para su equipo). Ejemplo de `.env`:

```
MONGODB_URI=mongodb://fulls:fulls@fullstack.dcc.uchile.cl:27019
MONGODB_DBNAME=fullstack
PORT=7001
HOST=0.0.0.0
```

- Ejecute `npm i` para instalar las dependencias en el servidor.

- Levante el servicio con PM2, que mantiene la aplicación corriendo aunque usted cierre la sesión SSH y la reinicia automáticamente si el proceso se cae:

```
pm2 start npm --name alanbritto-7001 -- run start
```

- El nombre del proceso (--name) debe ser único: use el nombre de su carpeta seguido de su puerto. Todos los grupos comparten el usuario fullstack, por lo que verán los procesos de los demás con `pm2 list`. No detenga, reinicie ni elimine procesos que no sean suyos.
- Guarde la lista de procesos para que el servicio se restablezca si el servidor se reinicia:

```
pm2 save
```

- Verifique que la página funciona accediendo a su URL. Por ejemplo: <https://fullstack.dcc.uchile.cl:7001/>, cambiando el puerto por el que le corresponda.

Comandos útiles de PM2 (siempre usando el nombre de su proceso):

Acción	Comando
Ver estado de sus procesos	<code>pm2 list</code>
Ver logs en vivo	<code>pm2 logs alanbritto-7001</code>
Reiniciar tras subir cambios	<code>pm2 restart alanbritto-7001</code>
Detener el servicio	<code>pm2 stop alanbritto-7001</code>
Eliminar el proceso de PM2	<code>pm2 delete alanbritto-7001</code>

Para verificar qué puerto se le asignó a cada integrante de su equipo, deberá ir a la sección Integrantes y revisar la numeración correspondiente a cada uno. El número asignado será su puerto con el prefijo 70. Ejemplo: Estudiante 17 → Puerto 7017